

Triggers - warsztaty

1. Utwórz tabelę Magazyn i Produkty i połącz je relacją.

```
create table Produkty
(idProduktu INT GENERATED ALWAYS AS identity primary key,
 idMagazynu int,
 nazwaProduktu varchar(50),
 cenaZakupu int )

create table Magazyn
(idMagazynu int primary key,
 Miasto varchar(50) )

alter table Produkty add foreign key (idMagazynu)
references Magazyn (idMagazynu)
```

2. Utwórz jedną procedurę, która umożliwi dodanie danych zarówno do tabeli Magazyn jak i Produkty:
 - a. Sprawdza, czy magazyn w podanym mieście istnieje
 - b. Jeśli nie, tworzy nowy magazyn z kolejnym dostępnym ID
 - c. Dodaje produkt do odpowiedniego magazynu
 - d. Obsługuje błędy
3. Nie pozwól dodać produktów jeśli cena będzie równa zero lub mniej. Zadbaj zarówno o *INSERT* jak i o *UPDATE*.

```
CALL dodaj_produkt_i_magazyn('Warszawa', 'Mysza', 0);
```

```
SQL Error [P0001]: BŁĄD: Wystąpił błąd: Nie można dodać produktu z
ceną poniżej zera. Podana cena: 0
Where: funkcja PL/pgSQL dodaj_produkt_i_magazyn(character
varying,character varying,integer), wiersz 30 w RAISE
```

4. Przygotuj kolejny trigger i tabelę logów, do której będzie odkładana informacja o każdym insercie i update z tabeli Magazyn wraz z datą. Należy odkładać zmianę ceny każdego produktu. Uwaga! Do tabeli logów trafia wpis tylko wtedy, gdy nowa cena jest wyższa od poprzedniej.

```
-- Tworzenie tabeli logów
CREATE TABLE LogiZmianCen (
```

```

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
idProduktu INT NOT NULL,
idMagazynu INT NOT NULL,
staraCena INT,
nowaCena INT NOT NULL,
dataZmiany TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
typOperacji VARCHAR(10) NOT NULL
);

```

```
CALL dodaj_produkt_i_magazyn('Poznań', 'Myszka', 10);
```

```
update produkty set cenazakupu = 99 where idproduktu = 5
```

```
select * from LogiZmianCen
```

123 id	123 idproduktu	123 idmagazynu	123 staracena	123 nowacena	datazmiany	A-Z typoperacji
1	5	2	[NULL]	10	2025-05-22 17:24:29.852	INSERT
2	5	2	10	99	2025-05-22 17:26:43.167	UPDATE

- Przygotuj trigger na tabeli magazyn, który nie pozwoli na update miasta. Zamiast tego ma pokazać komunikat iż update jest niemożliwy i należy dodać nowy magazyn

```
update Magazyn set miasto = 'Opole' where idmagazynu = 1;
```



SQL Error [P0001]: BŁĄD: Zmiana miasta dla magazynu jest niedozwolona. Miasto: Warszawa (ID: 1). Zamiast tego należy utworzyć nowy magazyn.

Where: funkcja PL/pgSQL blokuj_zmiane_miasta(), wiersz 5 w RAISE